

SemCast: One Representation Semantic Communication for Multi-Task Receivers

Sheng Yun[✉], *Student Member, IEEE*, and Ping Wang[✉], *Fellow, IEEE*

Abstract—Traditional wireless image transmission **emphasizes** bit-perfect data delivery, allocating resources to all bit-level information, including components that may be irrelevant to downstream task performance. This approach becomes increasingly inefficient as **emerging edge applications** such as autonomous driving, robotics, and remote sensing require task-level **semantic understanding** rather than pixel-level reconstruction. Existing semantic communication methods attempt to bridge this gap by transmitting task-relevant features, but they often train task-specific encoders and require channel-specific training, which hinders **generalizability** when applications and deployment conditions change. We present SemCast, a semantic communication system **centered on** unified representation transmission: a frozen Joint Embedding Predictive Architecture (JEPA) encoder produces a task-agnostic semantic representation that is transmitted once via joint source channel coding (JSCC) and then consumed by multiple downstream task heads at the receiver, each operating as **a function** of the recovered latent. The transmitter employs a learned mapper with unequal error protection (UEP) that automatically learns to allocate higher transmit power to task-critical feature dimensions under a transmit power constraint. At the receiver, PRISM-Rx (Pilotless Representation-based Implicit Semantic Mapper - Receiver) introduces a mixture-of-experts (MoE) demapper with low-rank adaptation (LoRA) and pilotless vector-quantized channel tokenization, a mechanism that extracts lightweight discrete summaries of channel conditions directly from received signals without pilot symbols, enabling Channel State Information (CSI)-free inference via conditional routing across diverse channels. We evaluate SemCast by quantifying end-to-end latent transmission fidelity under various channel conditions and by using classification as a representative downstream probe, while demonstrating that the same unified representation supports diverse tasks via lightweight heads. When **extending** to a new channel family, SemCast supports a commissioning procedure that uses a small labeled calibration set to add one new LoRA expert and fine-tune routing while freezing shared components, enabling parameter-efficient adaptation with minimal overhead. A lightweight compression stack, including low-bit quantization, further supports practical deployment under tight memory and bandwidth constraints.

Index Terms—semantic communication, joint source-channel coding (JSCC), representation learning, JEPA, CSI-free receiver, mixture-of-experts (MoE), LoRA, vector quantization, receiver adaptation

I. INTRODUCTION

Vision-centric workloads are becoming first-class citizens at the network edge, spanning diverse applications from autonomous driving and robotics to surveillance and remote sensing. Conventional wireless communication pipelines are optimized for bit-accurate reconstruction of packets, dedicating substantial bandwidth and latency budgets to preserve

all bit-level information, including components that may be irrelevant to downstream task performance [1]. However, when the objective is task execution rather than perfect data delivery, this separation between communication and inference becomes a significant bottleneck under tight bandwidth, latency, and energy constraints.

Semantic communication [2] has emerged to address this fundamental mismatch by transmitting task-relevant features instead of raw pixels. By operating directly in a learned feature space, semantic communication avoids transmitting information that is irrelevant to the downstream task, thereby reducing bandwidth consumption and improving task performance under constrained channel conditions. However, most existing systems based on Deep learning Joint Source Channel Coding (DeepJSCC) [3] still train task-specific encoders from scratch and typically assume the availability of Channel State Information (CSI), obtained at the cost of pilot signaling overhead. Such a design hinders **generalizability** along three axes: (i) it requires a dedicated encoder for each task or application, leading to duplicated model architectures and training effort; (ii) it necessitates retraining or fine-tuning whenever channel conditions change, tightly coupling the learned representation to a specific channel realization; and (iii) it offers limited support for multi-task inference, making it difficult to jointly support multiple tasks such as counting, attribute recognition, depth estimation, and segmentation from a single transmitted representation. Recent diffusion-based JSCC methods [4] improve reconstruction quality across various channels, yet they do not directly tackle unknown-channel adaptation for downstream semantic inference.

To address the aforementioned limitations, we propose SemCast, a semantic communication framework that explicitly decouples representation learning, channel transmission, and receiver adaptation, allowing each component to be independently designed and optimized. On the transmitter side, a frozen Joint Embedding Predictive Architecture (JEPA) encoder, pretrained with a masked prediction objective that **predicts** masked token representations from visible context **to learn** robust task-agnostic features, produces a single semantic representation that serves multiple tasks. This representation is then mapped to channel symbols by a learned JSCC module with Unequal Error Protection (UEP), operating under a transmit power constraint. On the receiver side, PRISM-Rx, Pilotless Representation-based Implicit Semantic Mapper - Receiver, employs a mixture-of-experts (MoE) demapper augmented with Low-Rank Adaptation (LoRA) and pilotless vector-quantized channel tokenization, a technique that discretizes received signal characteristics into compact tokens without requiring pilot signals (**details are provided in Sec-**

S. Yun and P. Wang are with the Department of Electrical Engineering and Computer Science, Lassonde School of Engineering, York University, Toronto, ON, Canada (e-mails: ys97@yorku.ca; pingw@yorku.ca).

tion V). This design enables CSI-free and implicit adaptation across diverse channels via conditional expert routing: each received sample is routed to a channel-specific expert based on its tokenized channel signature, allowing the receiver to handle varying channel conditions without explicit CSI. When deployment channels differ from those seen during training, SemCast supports parameter-efficient receiver adaptation by adding a new LoRA expert and fine-tuning the router and the newly added expert while keeping shared components frozen.

SemCast is best viewed as a unified representation transmission system: the transmitter sends a task-agnostic latent, and all downstream tasks are defined as functions of the recovered latent. Accordingly, our primary communication objective is robust latent recovery without CSI. If the recovered representation faithfully preserves the transmitted semantics, then any task head operating on it will produce accurate outputs. In this paper, we evaluate the receiver's robustness under diverse channel conditions in two complementary ways: (i) using classification as a representative downstream task to probe semantic performance, and (ii) directly quantifying latent transmission fidelity by measuring representation alignment between the recovered latent and the transmitted latent. We additionally demonstrate the breadth of tasks supported by the unified representation by attaching lightweight task-specific heads to support a diverse set of downstream tasks.

The main contributions of this work are as follows.

- **Unified representation transmission with learned JSCC and unequal error protection (UEP):** SemCast transmits a single pooled latent to significantly reduce communication overhead from a frozen JEPA encoder using an end-to-end JSCC mapper that learns UEP to prioritize task-critical dimensions, enabling robust multi-task inference and high latent fidelity across channels without CSI.
- **PRISM-Rx: pilotless, CSI-free reception via tokenization and expert routing:** PRISM-Rx achieves CSI-free demapping by combining pilotless vector-quantized (VQ) channel tokenization, Switch routing, and LoRA-augmented experts, where conditional computation routes each received sample to one selected expert to implicitly adapt to diverse channel conditions.
- **Parameter-efficient commissioning for unseen channels and deployment-oriented analysis:** SemCast supports commissioning on unseen channel families by adding a new LoRA expert and fine-tuning the router with a small labeled calibration set, which is distinct from the pilotless CSI-free inference path used over the training channel mixture, and we provide deployment-focused studies including ablations, mixed-channel robustness, and footprint/quantization analysis.

II. RELATED WORKS

Deep learning based JSCC parameterizes encoders and decoders as neural networks and optimizes them end-to-end with channel simulation. DeepJSCC [3] showed that learning-based transmission outperforms separation-based schemes at low Signal-to-Noise Ratio (SNR) and finite blocklength by jointly

optimizing source compression and channel coding. Subsequent work has extended this foundation in multiple directions. Attention mechanisms [5] were introduced to capture long-range dependencies in images, improving reconstruction quality. Semantic segmentation [6] was explored to enable task-specific transmission for dense prediction tasks. Bandwidth-adaptive transmission [3] allows dynamic adjustment of compression rates based on channel conditions. Multi-task learning [7] attempts to support multiple tasks but typically requires task-specific encoder branches. Wireless image retrieval [8] focuses on feature matching for content-based retrieval over wireless channels. Nonlinear transforms [9] improve rate-distortion performance through learned nonlinear mappings. Additional work has explored variable-rate transmission, unequal error protection, and joint optimization with multiple antennas [10]. Semantic communication frameworks [2], [11] emphasize preserving task-relevant information. However, most approaches still train task-specific encoders from scratch. Such encoders cannot generalize across multiple downstream tasks, and each task necessitates a dedicated encoder. As a result, supporting multiple tasks requires either repeated transmissions of different task-specific representations or costly retraining and reconfiguration of the encoder, significantly increasing communication overhead, inference latency, and system complexity.

To address these limitations, we utilize frozen, pre-trained JEPA encoders [12] that yield task-agnostic semantic representations, enabling a unified transmitted latent to serve multiple downstream tasks without retraining. This representation-centric design differs from prior task-specific semantic communication pipelines by decoupling representation learning from downstream heads and channel adaptation. Among general-purpose self-supervised vision encoders, contrastive DINOv2 [13], pixel-reconstruction MAE [14], and self-distillation Data2Vec [15] provide natural comparison points because they represent alternative encoder pretraining objectives under the same frozen-feature evaluation setting.

On the receiver side, traditional designs rely on pilot-based channel estimation and equalization [1], [16]. Recent learning-based alternatives include channel estimation networks [17] that predict CSI from pilot signals using neural networks, learned equalizers [18] that directly map received signals to transmitted symbols, end-to-end optimization [19] that jointly trains transmitter and receiver, meta-learning [20] that enables fast adaptation to new channel conditions with few samples, and online adaptation [21] that continuously updates receiver parameters during deployment to improve robustness. Nevertheless, most existing receivers rely on explicit CSI, which introduces pilot overhead and computational complexity, potentially limiting practical deployment. To overcome this challenge, we propose a receiver that integrates a Mixture-of-Experts (MoE) architecture [22] with pilotless vector-quantized channel signatures [23] and LoRA [24], thereby enabling blind adaptation across heterogeneous channel conditions without the need for CSI.

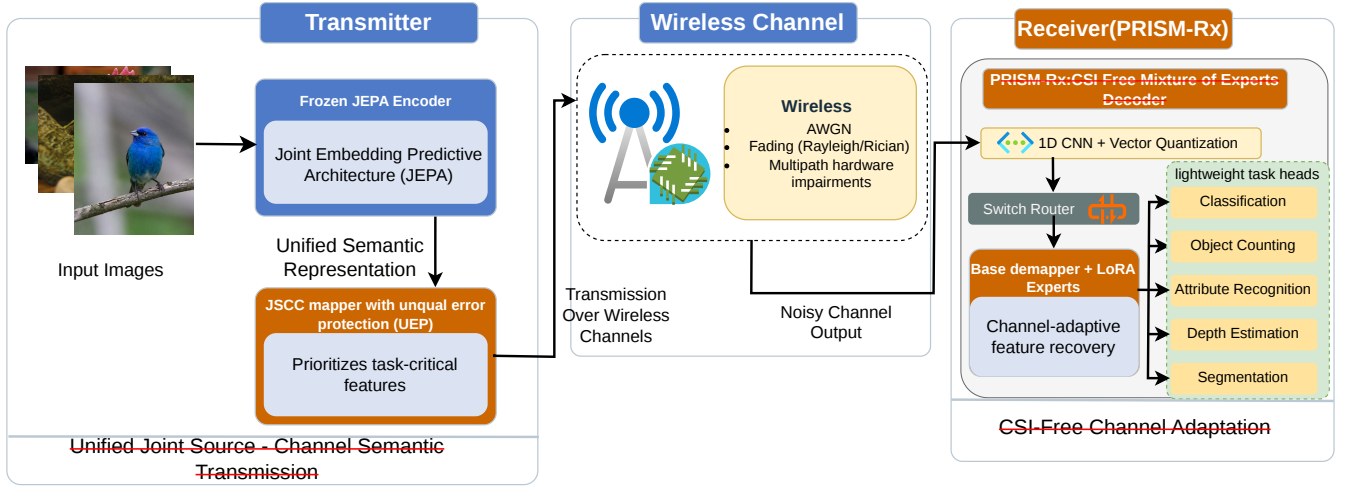


Fig. 1. System overview. **Transmitter:** input images are encoded by a frozen JEPA encoder to produce a unified, task-agnostic semantic representation; the pooled latent is mapped to channel symbols by a learned JSCC mapper with unequal error protection (UEP) and power normalization. **Wireless channel:** JSCC-encoded symbols are transmitted over diverse conditions (AWGN, Rayleigh/Rician fading, frequency-selective multipath, and hardware impairments), yielding a noisy channel output. **Receiver (PRISM-Rx):** a pilotless discrete channel tokenizer (1D CNN + VQ) extracts channel signatures for CSI-free adaptation; a Switch router selects an expert, and a shared base demapper with LoRA experts recovers the latent representation. **Multiple tasks at receiver:** lightweight task heads (e.g., classification, counting, attribute recognition, depth, segmentation) operate on the recovered representation.

III. PRELIMINARIES AND PROBLEM FORMULATION

A. System Model and Representation Learning

We consider the following semantic communication problem: can a single transmitted visual representation remain useful for multiple downstream tasks after passing through a wireless channel, without redesigning the transmitter for each task or relying on explicit CSI at the receiver? In SemCast, an input image $x \in \mathbb{R}^{H \times W \times 3}$ is processed by a frozen encoder to produce semantic features $Z = f_\theta(x) \in \mathbb{R}^{L \times D}$, where L denotes the number of spatial tokens and D is the feature dimension per token. We then compress these features into a compact global representation by mean-pooling, $\bar{z} = \frac{1}{L} \sum_{t=1}^L z_t \in \mathbb{R}^D$, and a learned mapper produces channel inputs $s \in \mathbb{R}^M$, where $M \ll LD$ denotes the number of transmitted symbols under the average transmit power constraint P_{avg} . The wireless channel yields received observations $y = c(s) + n$ with $n \sim \mathcal{N}(0, \sigma_n^2 \mathbf{I})$, and the receiver r_w must recover $\hat{z} = r_w(y)$ so that $\hat{z} \approx \bar{z}$ without explicit CSI. This setup focuses the communication problem on preserving one shared semantic interface that can later support multiple inference heads at the receiver.

The encoder is chosen to make this shared-latent formulation meaningful. We use a pretrained Joint Embedding Predictive Architecture (JEPA) encoder [12], which learns spatially grounded representations through masked prediction in latent space rather than pixel reconstruction. Given a visible-patch context, a predictor reconstructs latent targets produced by a momentum encoder on the full image. Let $\mathcal{M}$ denote the set of masked indices. With encoder f_θ , predictor p_ϕ , and momentum encoder $f_{\bar{\theta}}$, the JEPA pretraining objective is

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\bar{z}_i - z_i^{\text{target}}\|_2^2, \quad \bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta, \quad (1)$$

where $\bar{z}_i$ is the predictor output for masked token i , z_i^{target} is the corresponding target representation from the momentum encoder, and τ is the exponential moving-average update coefficient. We freeze this pretrained encoder during communication training so that the transmitted latent remains task-agnostic and reusable across downstream tasks, while the communication pipeline learns to preserve that latent through the channel rather than relearn the representation itself.

B. Receiver Side Multi-Task Inference

A key feature of SemCast is that it transmits a single task-agnostic representation from which all downstream tasks are derived. After PRISM-Rx recovers $\hat{z}$, lightweight receiver-side heads perform task inference. Classification, counting, and attribute recognition operate directly on the pooled representation $\hat{z}$, while dense prediction tasks such as segmentation and depth estimation require a spatial token map that preserves the per-patch layout before pooling. In this paper, we evaluate such dense heads as clean-feature transfer on Z , which lets us test whether the same encoder produces token-level representations rich enough for spatially detailed tasks while the transmission pipeline focuses on preserving the pooled shared latent.

The performance of any downstream task head operating on the received latent is determined by how accurately the reconstructed representation $\hat{z}$ matches the original $\bar{z}$. In particular, if a task head $h_k(\cdot)$ is L_k -Lipschitz in the representation space, then the task output deviation is bounded by $\|h_k(\hat{z}) - h_k(\bar{z})\| \leq L_k \|\hat{z} - \bar{z}\|$. This gives the central multi-task bridge used throughout the paper: when PRISM-Rx preserves the transmitted latent with high fidelity, every downstream head defined on that latent remains reliable up to a proportional perturbation bound. Accordingly, we investigate system robustness both through a representative downstream

probe (classification) and through explicit latent-fidelity metrics under channel impairments, while also reporting additional task heads to demonstrate the breadth of tasks supported by the unified representation.

C. Transmission Impairments

To evaluate whether the shared latent remains useful under realistic deployment conditions, we consider a broad range of wireless operating conditions, including additive white Gaussian noise (AWGN), flat-fading Rayleigh and Rician channels, frequency-selective multipath, and practical hardware impairments. AWGN captures pure additive corruption. Rayleigh and Rician fading are modeled as complex scalar gains applied to I/Q symbol pairs, with the Rician K-factor controlling the relative strength of the line-of-sight component versus scattered paths. Frequency-selective multipath is modeled using a finite impulse-response filter with random complex taps, so that different frequency components experience different fading. We also include carrier frequency offset, timing offset, IQ imbalance, and Rapp power-amplifier (PA) nonlinearity, all of which are common in practical radio hardware and can substantially distort the received signal before latent recovery.

The signal-to-noise ratio (SNR), a fundamental measure of transmission quality, is defined as $\text{SNR}_{\text{dB}} = 10 \log_{10}(P_{\text{avg}}/\sigma_n^2)$, where P_{avg} denotes the average transmit power and σ_n^2 represents the noise variance. Higher SNR values correspond to cleaner received signals, while lower SNR values indicate more severe noise corruption, making accurate latent recovery more challenging.

IV. SEMCAST ARCHITECTURE DESIGN

Figure 1 summarizes SemCast as an end-to-end semantic communication pipeline built around a single question: ~~can the transmitted visual representation remain useful for multiple downstream tasks after wireless transmission?~~ Starting from an input image x , the system extracts a shared semantic representation, compresses it for bandwidth-efficient transmission, sends it through an unknown channel, and reconstructs it at the receiver for downstream inference. The key architectural choice is that SemCast transmits one common latent rather than separate task-specific messages, so the communication pipeline is optimized to preserve a reusable semantic interface instead of specializing the transmitter for only one task.

On the transmitter side, a frozen JEPA encoder produces token features $\mathbf{Z} \in \mathbb{R}^{L \times D}$ that are summarized by mean-pooling into the compact latent $\bar{\mathbf{z}}$. A learned JSCC mapper then converts $\bar{\mathbf{z}}$ into M channel uses with unequal error protection, allowing the system to transmit semantic features directly rather than first converting them to a discrete bitstream. This design is chosen because the frozen encoder provides a stable task-agnostic representation that can be reused across downstream tasks, while JSCC offers graceful degradation under finite blocklength and noisy channels. In this way, SemCast separates representation learning from channel adaptation: the encoder defines what should be preserved, and the communication stack learns how to preserve it efficiently.

On the receiver side, PRISM-Rx recovers $\hat{\mathbf{z}}$ from the corrupted observation $\mathbf{y}$ without explicit CSI. Its operation follows the same high-level sequence throughout the paper: a **pilotless tokenizer** converts the received signal into compact channel signatures, a switch router selects one expert, and a shared demapper augmented by expert-specific LoRA updates reconstructs the transmitted latent. Duo-Track fusion and early-exit heads further improve robustness and confidence-aware inference. This decomposition is deliberate. Different channel regimes distort the signal in different ways, so SemCast uses conditional computation rather than forcing a single demapper to handle all cases equally well. When deployment channels drift beyond the original training mixture, the same receiver can be commissioned by adding a new LoRA expert and refining the router while keeping shared components fixed.

The recovered latent serves as the common interface for downstream task heads at the receiver, which is why the system supports classification, counting, attribute recognition, and related tasks without altering the transmitter. For practical deployment, SemCast also combines dimensionality reduction and low-bit quantization to reduce transmission bandwidth, memory footprint, and edge-side compute while preserving semantic utility. The remainder of the paper follows this division of roles: Section III formalizes the shared-latent problem and representation choice, Section V develops the detailed PRISM-Rx receiver design, and Section VII evaluates how well the overall system preserves multi-task semantic utility under realistic channel conditions.

V. PRISM-RX: CSI-FREE RECEIVER WITH CONDITIONAL COMPUTATION

This section details the internal components of PRISM-Rx, the receiver introduced in Section IV. PRISM-Rx recovers the transmitted semantic embedding without explicit CSI by combining a pilotless tokenizer, a switch router, and LoRA experts to perform **demapping** across diverse channel regimes. In SemCast, the recovered latent $\hat{\mathbf{z}}$ is the unified interface consumed by downstream task heads.

The receiver **operates in four stages**. First, the discrete channel tokenizer summarizes the received I/Q sequence into compact **channel tokens** without pilots. Second, a switch router converts those tokens into an expert decision. Third, the selected LoRA expert modulates a shared demapper that reconstructs the transmitted latent. Fourth, optional fusion and early-exit heads refine the recovered representation for downstream inference. ~~Presenting the receiver in this order mirrors the actual inference path and highlights why PRISM-Rx can remain CSI-free while still specializing across heterogeneous channel conditions.~~

A. Motivation

Traditional JSCC receivers employ a single demapping network trained to operate across channel conditions. However, the optimal demapping function varies significantly with channel characteristics. AWGN channels are well addressed by simple linear equalization, while multipath channels require

sophisticated interference mitigation, and hardware impairments demand **nonlinear distortion compensation**. Training a single network to handle all cases leads to suboptimal specialization.

An alternative approach uses separate receiver models for different channel types, switching between them based on CSI or blind channel estimation. However, this requires explicit channel classification, increases memory footprint linearly with the number of channel types, and struggles with intermediate or compound channel effects.

PRISM-Rx adopts a mixture-of-experts paradigm that achieves the specialization benefits of multiple receivers while maintaining compact parameterization and avoiding explicit channel classification. The key insight is to let the routing network implicitly discover channel condition clusters from received signal characteristics, and equip each expert with low-rank adapters that **specialize** in a shared base receiver. This design enables fine-grained adaptation with minimal parameter overhead. **The tokenizer-router-expert decomposition therefore serves two goals simultaneously: it removes pilot overhead during inference and concentrates model capacity where different channel realizations actually require different demapping behavior.**

B. Discrete Channel Tokenizer (DCT)

The first component of PRISM-Rx is a pilotless channel tokenizer that converts the received signal $\mathbf{y} \in \mathbb{R}^M$ into a discrete token sequence. Unlike conventional pilot-based channel estimation that requires dedicated transmission resources, our tokenizer learns to capture channel effects directly from the received signal structure. We call this a tokenizer because it discretizes continuous received signal features into discrete tokens via vector quantization, similar to how text tokenizers discretize words.

1) *Signal representation*: A received vector $\mathbf{y}$ is the length- M real-valued observation produced by the physical channel over M channel uses. We interpret $\mathbf{y}$ as interleaved in-phase and quadrature samples corresponding to $L_s = M/2$ complex baseband symbols, and we build local windows over this 1D symbol sequence. Windowing is needed to capture local signal structure: by segmenting the received signal into overlapping blocks, we can extract features that characterize channel effects at different positions in the transmission. The window size (128 symbols) balances capturing sufficient context while maintaining computational efficiency. In this abstraction, the term “subcarrier” refers to the index within a window (i.e., a position along the channel-use sequence) rather than a literal OFDM subcarrier; our tokenizer is agnostic to any specific multicarrier waveform and operates directly on the received sample sequence.

2) *I/Q Windowing*: We interpret the received vector as interleaved in-phase (I) and quadrature (Q) samples forming complex baseband symbols. By “windowing,” we mean segmenting the signal into overlapping blocks of **128 symbols** with **64-symbol** hop size (50% overlap), enabling local feature extraction while maintaining continuity across the sequence.

Consecutive pairs (y_{2i}, y_{2i+1}) are grouped into a **two-stream** sequence:

$$\mathbf{Y}_{\text{IQ}} = \begin{bmatrix} y_0 & y_2 & y_4 & \cdots \\ y_1 & y_3 & y_5 & \cdots \end{bmatrix} \in \mathbb{R}^{2 \times L_s}, \quad (2)$$

where $L_s = M/2$ is the number of complex symbols. This sequence is divided into overlapping windows of length **$W = 128$** with hop size **$W/2 = 64$** , yielding N_w windows:

$$\{\mathbf{Y}_{\text{IQ}}^{(1)}, \mathbf{Y}_{\text{IQ}}^{(2)}, \dots, \mathbf{Y}_{\text{IQ}}^{(N_w)}\}, \quad \mathbf{Y}_{\text{IQ}}^{(t)} \in \mathbb{R}^{2 \times W}. \quad (3)$$

3) *Token Encoder*: Each window is processed by a lightweight convolutional encoder to produce per-position latent codes (one code per symbol index within the window). We denote this encoder by $\text{Enc}_{\text{DCT}}(\cdot)$ and use $\mathbf{E}^{(t)}$ to represent the matrix of per-position latent codes produced for window t . We use the Gaussian Error Linear Unit (GELU) activation throughout the tokenizer:

$$\mathbf{E}^{(t)} = \text{Enc}_{\text{DCT}}(\mathbf{Y}_{\text{IQ}}^{(t)}) \in \mathbb{R}^{W \times D_c}, \quad (4)$$

where $D_c = 64$ is the code dimension. The encoder consists of three 1D convolutional layers operating along the symbol-index axis:

$$\mathbf{H}_1 = \text{GELU}(\text{Conv1D}_{2 \rightarrow 128}(\mathbf{Y}_{\text{IQ}}^{(t)})), \quad (5)$$

$$\mathbf{H}_2 = \text{GELU}(\text{Conv1D}_{128 \rightarrow 128}(\mathbf{H}_1)), \quad (6)$$

$$\mathbf{E}^{(t)} = \text{Conv1D}_{128 \rightarrow D_c}(\mathbf{H}_2). \quad (7)$$

The final layer is linear, without GELU, so that the encoder can produce unconstrained latent codes before vector quantization.

4) *Vector Quantization*: Vector Quantization (VQ) [23] discretizes continuous latents using a learned codebook $\mathcal{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_K\}$ by nearest-neighbor assignment. For each continuous code $\mathbf{e}$ **generated by the token encoder**, VQ finds the closest codebook entry: $\mathbf{e}^q = \mathbf{c}_{k^*}$ where $k^* = \arg \min_k \|\mathbf{e} - \mathbf{c}_k\|_2^2$. Discrete tokens are robust, compact summaries used for routing.

VQ is **needed** to create discrete, robust channel signatures: continuous features are sensitive to small perturbations, while discrete tokens provide stable, categorical representations of channel states that enable effective routing. The continuous latent codes are discretized using this learned codebook with **$K = 256$** entries. For each window position code $\mathbf{e}_w \in \mathbb{R}^{D_c}$, we find the nearest codebook entry:

$$k_w^* = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{e}_w - \mathbf{c}_k\|_2^2. \quad (8)$$

The quantized representation is:

$$\mathbf{e}_w^q = \mathbf{c}_{k_w^*}, \quad (9)$$

with straight-through gradient estimator for backpropagation: $\nabla_{\mathbf{e}_w} \mathbf{e}_w^q = \mathbf{I}$.

The VQ training objective combines **a codebook loss, which updates the codebook entries toward encoder outputs, and a commitment loss, which encourages encoder outputs to stay**

close to their selected codebook entries:

$$\mathcal{L}_{VQ} = \underbrace{\beta \|\text{sg}[\mathbf{E}^{(t)}] - \mathbf{E}_q^{(t)}\|^2}_{\text{codebook loss}} + \underbrace{\|\mathbf{E}^{(t)} - \text{sg}[\mathbf{E}_q^{(t)}]\|^2}_{\text{commitment loss}}, \quad (10)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operation, which blocks gradient flow through its argument during backpropagation, $\mathbf{E}_q^{(t)}$ is the quantized code matrix, and $\beta = 0.25$ is the codebook-loss weight. By keeping encoder outputs close to the selected codebook entries, the commitment loss reduces unstable switching among entries and thereby stabilizes codebook usage during training.

5) *Self-Supervised Auxiliary Tasks*: We train the tokenizer with two auxiliary self-supervised losses: the masked prediction loss $\mathcal{L}_{\text{mp}}$ and the contrastive predictive coding loss $\mathcal{L}_{\text{CPC}}$. These losses are called self-supervised because they do not require labeled data; instead, they use the structure of the received signal itself as supervision. The masked prediction loss encourages the tokenizer to capture sufficient information to reconstruct the signal, while the contrastive predictive coding loss encourages temporal consistency. First, a masked prediction loss trains a small decoder to recover masked I/Q samples from quantized codes:

$$\mathcal{L}_{\text{mp}} = \|\mathbf{M} \odot (\mathbf{Y}_{\text{IQ}}^{(t)} - \text{Dec}_{\text{PCT}}(\mathbf{E}_q^{(t)}))\|^2 + \mathcal{L}_{VQ}, \quad (11)$$

where $\mathbf{M}$ masks a random fraction (e.g., 15%) of I/Q entries. Second, a contrastive predictive coding (CPC) loss encourages temporal consistency across windows. We pool codes within each window to obtain window-level representations $\bar{\mathbf{e}}_t = \frac{1}{W} \sum_{w=1}^W \mathbf{e}_{t,w}^q$, and train a predictor to predict future windows:

$$\hat{\mathbf{e}}_{t+k} = \mathbf{W}_{\text{CPC}} \bar{\mathbf{e}}_t, \quad k \in \{1, 2, 3\}, \quad K_{\text{pred}} = 3. \quad (12)$$

The CPC loss uses InfoNCE (information noise-contrastive estimation) and is defined as follows:

$$\mathcal{L}_{\text{CPC}} = -\frac{1}{K_{\text{pred}}} \sum_{k=1}^{K_{\text{pred}}} \log \frac{\exp(\hat{\mathbf{e}}_{t+k}^T \bar{\mathbf{e}}_{t+k} / \tau)}{\sum_{t'} \exp(\hat{\mathbf{e}}_{t+k}^T \bar{\mathbf{e}}_{t'} / \tau)}, \quad (13)$$

where $\tau = 0.07$ is the temperature and the sum in the denominator ranges over all windows in the batch.

The tokenizer outputs a discrete token matrix $\mathbf{I} \in \{1, \dots, K\}^{N_w \times W}$ summarizing channel effects on the received signal.

C. Switch Router with Load Balancing

The token indices $\mathbf{I}$ are fed to a switch router that assigns each sample to one of N_e experts, where each expert uses distinct LoRA parameters. Rather than routing based on individual tokens, we compute a histogram representation that captures the overall token distribution:

$$\mathbf{g}_b = \frac{1}{N_w \cdot W} \sum_{t=1}^{N_w} \sum_{w=1}^W \mathbf{1}_{I_{t,w}} \in \mathbb{R}^K, \quad (14)$$

where $\mathbf{1}_k$ is a one-hot vector with 1 at position k . This histogram provides a compact, order-invariant summary of channel characteristics.

Equivalently, $\mathbf{1}_k$ is a binary vector whose k -th entry is 1 and all other entries are 0.

A small multi-layer perceptron (MLP) maps the histogram to expert assignment logits:

$$\ell_b = \text{MLP}_{\text{route}}(\mathbf{g}_b). \quad (15)$$

The MLP has one hidden layer with 128 units and GELU activation.

During training, we employ stochastic top-1 routing by sampling one expert index from the softmax distribution:

$$p_i = \frac{\exp(\ell_i)}{\sum_{j=1}^{N_e} \exp(\ell_j)}, \quad i^* \sim \text{Categorical}(\mathbf{p}). \quad (16)$$

At inference, we use deterministic routing: $i^* = \arg \max_i \ell_i$. Here, $\text{Categorical}(\mathbf{p})$ denotes a discrete distribution over experts with probabilities given by $\mathbf{p}$.

1) *Load Balancing Loss*: To prevent all samples from routing to a single dominant expert (which would defeat the purpose of having multiple experts and reduce model capacity), we introduce a load-balancing auxiliary loss that encourages the average routing distribution to match a target load t (uniform when unspecified). Let p_i denote the average routing probability for expert i over the batch. The loss is a Kullback-Leibler (KL) divergence between the realized load and the target:

$$\mathcal{L}_{\text{bal}} = \sum_{i=1}^{N_e} p_i (\log p_i - \log t_i), \quad t_i = \frac{1}{N_e}. \quad (17)$$

This formulation encourages experts to specialize in distinct sample subsets while maintaining balanced utilization. The rationale is that balanced expert usage ensures all experts are trained effectively and that the model's capacity is fully utilized, while still allowing experts to specialize in different channel conditions through the routing mechanism.

2) *Router Z-Loss*: Large router logits, that is, the pre-softmax expert scores, can cause training instability. The z-loss regularizes the logit magnitudes via a mean-squared penalty:

$$\mathcal{L}_z = \lambda_z \cdot \frac{1}{BN_e} \sum_{b=1}^B \sum_{i=1}^{N_e} (\ell_i^{(b)})^2, \quad (18)$$

where B is the batch size, N_e is the number of experts, and $\lambda_z = 10^{-5}$ is a small scaling factor.

The total router loss is:

$$\mathcal{L}_{\text{router}} = \mathcal{L}_{\text{bal}} + \mathcal{L}_z. \quad (19)$$

D. Base Receiver with LoRA Experts

Mixture-of-Experts (MoE) [22] routes inputs to specialized subnetworks, called experts, via a gating network, improving model capacity with conditional computation, meaning that each input activates only the selected expert rather than every expert. Top-1 routing, also known as switch routing, assigns one expert per input and requires load balancing to avoid collapse. Low-Rank Adaptation (LoRA) [24] injects trainable rank- r matrices into frozen linear layers. Specifically, for a

frozen weight matrix $\mathbf{W}_0$, LoRA augments the transformation as

$$\mathbf{y} = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B}(\mathbf{A}^T \mathbf{x}), \quad \mathbf{A} \in \mathbb{R}^{d_{in} \times r}, \mathbf{B} \in \mathbb{R}^{d_{out} \times r}, \quad (20)$$

where $\mathbf{A} \in \mathbb{R}^{d_{in} \times r}$ and $\mathbf{B} \in \mathbb{R}^{d_{out} \times r}$ are low-rank matrices with rank $r \ll \min(d_{in}, d_{out})$, and α is a scaling hyperparameter. This design achieves parameter-efficient specialization: only the small matrices $\mathbf{A}$ and $\mathbf{B}$ are trained, while $\mathbf{W}_0$ remains frozen, preserving pretrained knowledge while enabling adaptation. In our setting, **routing conditioned on tokenizer-derived channel signatures** enables **CSI-free robustness across diverse channels** by selecting specialized experts, and LoRA provides a lightweight mechanism for explicit deployment-time **adaptation** by **adding or updating** a small number of parameters.

The core demapping network consists of **a lightweight base receiver that is adapted per-expert through low-rank adapters**. The base receiver **is shared across all experts** and provides a common foundation, while LoRA adapters add expert-specific modifications with minimal parameters. The base receiver is a two-layer MLP that maps the received signal $\mathbf{y} \in \mathbb{R}^M$ to the semantic feature space $\mathbb{R}^D$:

$$\mathbf{h} = \text{GELU}(\mathbf{W}_1 \mathbf{y} + \mathbf{b}_1), \quad \mathbf{h} \in \mathbb{R}^{D_h}, \quad (21)$$

$$\mathbf{h}' = \text{LN}_{\text{non-affine}}(\mathbf{h}), \quad (22)$$

$$\hat{\mathbf{z}} = 0.1 \cdot (\mathbf{W}_2 \mathbf{h}' + \mathbf{b}_2), \quad \hat{\mathbf{z}} \in \mathbb{R}^D, \quad (23)$$

where $D_h = 512$ is the hidden dimension and $\text{LN}(\cdot)$ denotes layer normalization. We employ non-affine layer normalization (without learnable scale and shift) to stabilize feature magnitudes without introducing additional learnable parameters that could interfere with LoRA adaptation. The output scaling factor 0.1 prevents excessively large feature norms that can destabilize downstream task heads.

For sample b assigned to expert i_b^* , we apply expert-specific **LoRA updates** to both linear layers. For the first layer:

$$\mathbf{h}_b = \text{GELU} \left(\mathbf{W}_1 \mathbf{y}_b + \mathbf{b}_1 + \frac{\alpha}{r} \mathbf{B}_1^{(i_b^*)} (\mathbf{A}_1^{(i_b^*)})^T \mathbf{y}_b \right), \quad (24)$$

where $\mathbf{A}_1^{(i)} \in \mathbb{R}^{M \times r}$ projects the input to a low-rank subspace, $\mathbf{B}_1^{(i)} \in \mathbb{R}^{D_h \times r}$ projects back to the hidden dimension, with rank $r = 8$, and we set $\alpha = r$ (so $\alpha/r = 1$) in our implementation. The scaling factor α/r controls the magnitude of the LoRA adaptation relative to the base weights. For the second layer, $\mathbf{A}_2^{(i)} \in \mathbb{R}^{D_h \times r}$ and $\mathbf{B}_2^{(i)} \in \mathbb{R}^{D \times r}$ follow the same pattern with appropriate input/output dimensions. Similarly, for the second layer:

$$\hat{\mathbf{z}}_b = 0.1 \cdot \left(\mathbf{W}_2 \mathbf{h}'_b + \mathbf{b}_2 + \frac{\alpha}{r} \mathbf{B}_2^{(i_b^*)} (\mathbf{A}_2^{(i_b^*)})^T \mathbf{h}'_b \right). \quad (25)$$

The LoRA matrices are initialized with $\mathbf{A}$ using Kaiming uniform initialization, and $\mathbf{B}$ is initialized to zeros, ensuring that the adapted model starts from the base model's behavior. As training progresses, each expert's LoRA parameters specialize to the channel conditions it is assigned.

This design achieves expert specialization with minimal parameter overhead. The parameter efficiency comes from the low-rank factorization: instead of learning full weight

matrices for each expert, we only learn small rank- r matrices. For $N_e = 5$ experts, the LoRA parameters add only $5 \times 2 \times (M \cdot r + D_h \cdot r + D \cdot r)$ parameters compared to $5 \times (M \cdot D_h + D_h \cdot D)$ for separate full networks.

E. Duo-Track Feature Fusion

The base receiver outputs are processed through a duo-track fusion module that combines classical signal processing with neural adaptation. This design provides a regularization pathway that prevents **pathological demapping behaviors**.

The classical track applies a lightweight MLP to the base receiver output:

$$\mathbf{z}_{\text{classical}} = \text{MLP}_{\text{classical}}(\text{LN}(\hat{\mathbf{z}})). \quad (26)$$

This track provides a stable, regularized pathway that helps prevent degenerate solutions.

The neural track is a copy of the base receiver output:

$$\mathbf{z}_{\text{neural}} = \hat{\mathbf{z}}. \quad (27)$$

This track preserves the direct expert-adapted features.

These tracks are fused using a multi-head cross-attention mechanism where the classical track serves as the query and the neural track provides keys and values:

$$\mathbf{Q} = \text{LN}_q(\mathbf{z}_{\text{classical}}), \quad (28)$$

$$\mathbf{K}, \mathbf{V} = \text{LN}_{kv}(\mathbf{z}_{\text{neural}}), \quad (29)$$

$$\mathbf{z}_{\text{attn}} = \text{MultiHeadAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}), \quad (30)$$

$$\mathbf{z}_{\text{fused}} = \text{LN}_{\text{non-affine}}(\mathbf{W}_{\text{proj}} \mathbf{z}_{\text{attn}} + \mathbf{z}_{\text{classical}}). \quad (31)$$

We use 2 attention heads. The final non-affine layer normalization ensures that the fused feature norms remain close to $\sqrt{D}$ regardless of attention amplification, which is critical for stable task head training.

F. Early-Exit Task Heads

PRISM-Rx includes **dual classification heads** that **enable early exiting** based on prediction confidence. The fast head is a single linear layer:

$$\mathbf{l}_{\text{fast}} = \mathbf{W}_{\text{fast}} \text{LN}(\mathbf{z}_{\text{fused}}) + \mathbf{b}_{\text{fast}} \in \mathbb{R}^C, \quad (32)$$

and the strong head is a two-layer MLP:

$$\mathbf{l}_{\text{strong}} = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \text{LN}(\mathbf{z}_{\text{fused}}) + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^C. \quad (33)$$

At inference, we first evaluate the fast head and compute prediction confidence:

$$\text{conf} = \max_c \text{softmax}(\mathbf{l}_{\text{fast}})_c. \quad (34)$$

If $\text{conf} \geq \tau_{\text{exit}}$ (typically $\tau_{\text{exit}} = 0.9$), we output the fast prediction; otherwise we output the strong prediction. When the fast head is highly confident, the sample is likely easy and does not require the additional capacity of the strong head. For uncertain samples, the strong head provides more robust predictions. **Accordingly, we view early-exit primarily as a confidence-based selection mechanism that can improve robustness and calibration under channel uncertainty. In the current implementation, both heads are still evaluated, and the gate only selects which output is used.**

Both heads are trained jointly during PRISM-Rx training with strong-head supervision and auxiliary terms:

$$\mathcal{L}_{\text{heads}} = \mathcal{L}_{\text{CE}}(\ell_{\text{strong}}, c) + \lambda_{\text{exit}} \mathcal{L}_{\text{CE}}(\ell_{\text{fast}}, c) + \lambda_{\text{kd}} \mathcal{L}_{\text{kd}}, \quad (35)$$

where c is the ground-truth class label, $\mathcal{L}_{\text{CE}}$ is cross-entropy loss, and $\mathcal{L}_{\text{kd}}$ is a knowledge distillation loss that transfers the strong-head soft predictions to the fast head:

$$\mathcal{L}_{\text{kd}} = T^2 \text{KL}(\text{softmax}(\ell_{\text{strong}}/T) \parallel \text{softmax}(\ell_{\text{fast}}/T)), \quad (36)$$

where T is the distillation temperature and $\text{KL}(\cdot \parallel \cdot)$ denotes the Kullback-Leibler divergence.

G. Training Procedure

PRISM-Rx is trained in multiple stages. In Stage S0, we pretrain the discrete channel tokenizer in a self-supervised manner on received signals using masked-subcarrier prediction and contrastive predictive coding (CPC), with the objective

$$\mathcal{L}_{\text{S0}} = \mathcal{L}_{\text{mp}} + 0.5 \mathcal{L}_{\text{CPC}}. \quad (37)$$

In Stage S1, we freeze the tokenizer and train the mapper, router, LoRA experts, fusion module, and task heads end-to-end with classification supervision under a Conditional Value at Risk distributionally robust optimization (CVaR-DRO) objective over multiple channel realizations per batch. In Stage S2, we continue training the same components under the same objective, while refining the model for improved robustness. The CVaR-DRO objective emphasizes difficult channel realizations by combining the mean classification loss with a tail-loss term over the worst realizations in each batch. This training objective aggregates the strong-head cross-entropy across channel realizations and adds router regularization and auxiliary head terms, including the weighted fast-head loss and knowledge distillation from the strong head to the fast head.

Initialization from Pretrained Heads: We initialize the early-exit heads from pretrained task heads trained on clean JEPA features. For the fast head, we compose the weights of a two-layer head $\mathbf{W}_{\text{clf}} = \mathbf{W}_2 \mathbf{W}_1$ to form a single effective layer. For the strong head, we directly transfer the MLP weights with truncation or zero-padding if the dimensions mismatch. This warm-start strategy provides an informative initialization that is then fine-tuned for robustness to channel impairments.

VI. UNKNOWN CHANNEL CONDITION ADAPTATION

Real deployments inevitably encounter channel conditions outside the training distribution: new propagation environments or composite effects (e.g., multipath coupled with hardware impairments). Although PRISM-Rx is designed for pilot-less, CSI-free inference over heterogeneous channel mixtures, a persistent distribution shift may necessitate an additional commissioning step. We therefore introduce a parameter-efficient receiver commissioning mechanism that uses a small labeled calibration set from the deployment environment to expand PRISM-Rx with augmented capacity while keeping shared components frozen.

A. Adaptation

1) Problem Statement: Let $\mathcal{D}_{\text{train}}$ denote the mixture of training channels (AWGN, Rayleigh, Rician, multipath, hardware impairments) used to train PRISM-Rx. In our adaptation implementation, we construct a small adaptation set from an unseen channel distribution $\mathcal{D}_{\text{new}}$ by sampling images, computing clean pooled semantic representations $\bar{\mathbf{z}}_{\text{clean}}$ from the frozen JEPA encoder, mapping them to transmitted vectors $\mathbf{x}_{\text{clean}}$ via the frozen mapper, and observing received vectors $\mathbf{y}$ from the unseen channel. This yields tuples $\{(\mathbf{y}^{(i)}, \mathbf{x}_{\text{clean}}^{(i)}, \bar{\mathbf{z}}_{\text{clean}}^{(i)})\}_{i=1}^N$ together with the corresponding class labels used for supervised adaptation. The objective is to obtain adapted receiver parameters that improve task performance on $\mathcal{D}_{\text{new}}$ while (i) freezing the tokenizer, base demapper, and existing experts to avoid catastrophic forgetting, and (ii) minimizing the additional parameter and computation overhead. We use $\bar{\mathbf{z}}_{\text{clean}}$ to denote the same task-agnostic representation that is transmitted in SemCast (i.e., the mean-pooled JEPA feature extracted from the calibration images prior to channel corruption). This clean reference provides an explicit calibration target for representation recovery on $\mathcal{D}_{\text{new}}$.

2) Method:

a) Trainable components: The adaptation operates within PRISM-Rx's MoE design and consists of two trainable components. First, expert expansion (LoRA) adds a new expert E_{new} realized as low-rank adapters injected into the base receiver's linear layers. For a frozen weight $\mathbf{W}_0$, LoRA adds trainable matrices $\mathbf{A} \in \mathbb{R}^{d_{\text{in}} \times r}$ and $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$ (rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$) with output

$$\mathbf{W}(\mathbf{x}) = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B} (\mathbf{A}^T \mathbf{x}). \quad (38)$$

We use Kaiming-uniform initialization for $\mathbf{A}$ and zero initialization for $\mathbf{B}$ so that the adapted path starts from the base receiver behavior, ensuring training stability. Second, router fine-tuning updates the switch router, which consumes the histogram signature of tokenizer VQ tokens and outputs logits ℓ over experts. We fine-tune its parameters to assign samples from $\mathcal{D}_{\text{new}}$ to E_{new} while preserving load balance across existing experts.

b) Optimization Objective: Given adaptation minibatches $\{(\mathbf{y}_b, c_b)\}_{b=1}^B$, we optimize a composite loss as follows:

$$\begin{aligned} \mathcal{L}_{\text{adapt}} = & \underbrace{\mathcal{L}_{\text{CE}}(h(\hat{\mathbf{z}}), c)}_{\mathcal{L}_{\text{task}}} + \underbrace{\lambda_{\text{feat}} \mathcal{L}_{\text{feat}}(\hat{\mathbf{z}}, \bar{\mathbf{z}}_{\text{clean}})}_{\text{feature consistency}} \\ & + \underbrace{\lambda_{\text{router}} \mathcal{L}_{\text{router-sup}}(\ell)}_{\text{router supervision}} + \underbrace{\lambda_{\text{lb}} \mathcal{L}_{\text{router}}}_{\text{load balance + z-loss}} \\ & + \underbrace{\lambda_{\text{kd}} \mathcal{L}_{\text{kd}}}_{\text{strong} \rightarrow \text{fast KD}}. \end{aligned} \quad (39)$$

where $\mathcal{L}_{\text{task}}$ is the classification loss, $\mathcal{L}_{\text{feat}}$ is a cosine-distance feature consistency term computed on normalized features, $\mathcal{L}_{\text{router-sup}}$ is a cross-entropy router-supervision term that encourages assignment to the newly added expert, $\mathcal{L}_{\text{router}}$ is the router auxiliary loss (load-balancing and z-loss) defined in Section V, and $\mathcal{L}_{\text{kd}}$ distills the strong head into the fast head. The clean pooled JEPA representation $\bar{\mathbf{z}}_{\text{clean}}$ provides

the calibration target for representation recovery on the same image.

c) *Training Protocol*: We freeze the tokenizer, base receiver, task heads, and all existing experts and only train E_{new} 's LoRA parameters and the router parameters using a small learning rate and early stopping when desired.

d) *Parameter Efficiency*: The LoRA path adds $\mathcal{O}(r(d_{\text{in}} + d_{\text{out}}))$ parameters per adapted layer. Because only the router head and low-rank matrices are updated, computation and memory overhead during adaptation and inference are minimal. Existing experts remain available for previously seen channels.

e) *Integration with PRISM-Rx*: Adaptation operates strictly on the PRISM-Rx architecture described in Section V. After the original staged PRISM-Rx training (Stage S0 followed by Stages S1/S2), the tokenizer and base demapper remain frozen. When an unseen channel is detected, we spawn E_{new} and fine-tune the router as above. Inference remains unchanged: the router selects the top-1 expert (often E_{new} on $\mathcal{D}_{\text{new}}$), and the selected expert's LoRA path specializes demapping.

f) *Default CSI-free robustness vs commissioning*: Adaptation complements PRISM-Rx's CSI-free robustness. In the default deployment path, the router selects a top-1 expert per sample based on tokenizer signatures, providing conditional computation and specialization without explicit CSI. This default behavior is not a separate adaptation stage; rather, it is the receiver's standard CSI-free inference mechanism over the training channel mixture. When an unseen channel emerges, receiver commissioning can be realized by adding a new LoRA expert and fine-tuning the router to preferentially route samples to it while keeping shared components frozen. For diagnostic purposes, we can force routing to the newly added expert to isolate the contribution of the adapted expert from routing behavior.

VII. EXPERIMENTS AND RESULTS

We evaluate SemCast along two complementary axes. First, we quantify the multi-task utility of the frozen JEPA representation by training lightweight downstream heads on clean features across diverse tasks. Second, we assess end-to-end semantic transmission robustness by mapping the pooled latent $\bar{z}$ through the JSCC link and measuring how accurately the CSI-free receiver recovers $\hat{z}$ under heterogeneous channel and hardware impairments. We use ImageNet classification as a representative downstream probe and report latent-fidelity metrics between $\hat{z}$ and $\bar{z}$. Finally, we assess PRISM-Rx by comparing against simplified receiver baselines and component ablations. We present the section in the same order as the claims developed in the paper: experimental setup first, then encoder selection, then PRISM-Rx behavior under known and unknown channels, and finally deployment-oriented complexity and compression analyses.

A. Experimental Setup

We organize the setup around datasets and tasks, baseline systems, and training configuration so that the later results can be interpreted in a consistent order.

1) *Datasets and Tasks*: We evaluate the system on five primary tasks. While the SemCast architecture enables one transmitted representation to support multiple tasks, we evaluate each task independently on its benchmark dataset so that each task can be assessed with its standard protocol while the same frozen encoder and communication pipeline are used throughout.

Image Classification: We use ImageNet-1K [25] with 1000 classes. Images are resized to 128×128 for efficient processing while preserving semantic content. The training set contains approximately 1.28M images, and the validation set contains 50K images. We report top-1 accuracy as the primary metric. For encoder comparison, we also report macro-averaged F1 score to capture per-class performance balance.

In our implementation, we load images at 128×128 ("ImageNet-128") for data efficiency, but the frozen ViT-H/14 encoder internally upsamples inputs to 224×224 before tokenization. This yields a 14×14 patch grid and $L = 196$ tokens (patch size 14), which we use consistently for latent-size calculations. For dense tasks (Cityscapes, NYU-D), we also upsample input images to 224×224 before the frozen encoder to maintain the same $L = 196$ token grid, then apply task-specific heads to the spatial token maps before pooling.

Depth Estimation: NYU Depth V2 [26] provides 1449 indoor RGB-D image pairs. We consider the official train/validation split and resize images to 128×128 . Depth maps are normalized to $[0, 1]$ range. We report Root Mean Square Error (RMSE) as the metric.

Object Counting: The CLEVR dataset [27] provides synthetic images with 3-10 objects per scene, along with detailed annotations. We use the count prediction task, where the goal is to predict the total number of objects. We report mean absolute error (MAE) on the count prediction.

Attribute Recognition: Also from CLEVR [27], we evaluate fine-grained attribute prediction (e.g., object color, shape, size) to demonstrate JEPA's capability for detailed semantic understanding. Metrics include presence F1 (whether an attribute is present in the image) and count MAE (error in predicting attribute counts).

Semantic Segmentation: Cityscapes [28] provides street scene images with per-pixel semantic annotations for 19 classes. Images are resized to 512×512 to preserve fine-grained spatial structure. Performance is reported using the mean Intersection over Union (mIoU) across all classes.

Unless stated otherwise, our end-to-end evaluation under channel impairments uses ImageNet classification as a representative downstream probe and reports latent fidelity ($\hat{z}$ vs. $\bar{z}$). Since all downstream tasks are functions of the recovered latent, high latent fidelity provides direct evidence that the same transmission can support multiple heads. We additionally report clean-image multi-task performance to demonstrate the breadth of tasks supported by the unified representation.

2) *Baseline Systems*: We compare against baseline receiver variants implemented in our evaluation scripts to isolate the benefits of different components of PRISM-Rx:

Mono (Single Expert): Forces all samples to use a single expert (top-1 routing disabled) while retaining the expert

TABLE I

PRETRAINED ENCODER COMPARISON ON CLEAN FEATURES. METRICS: TOP-1 AND F1 (ImageNet-1K), MAE (CLEVR COUNTING), PRESENCE F1 AND COUNT MAE (CLEVR ATTRIBUTES), RMSE (NYU-DEPTH V2), AND mIoU (CITYSCAPES). ARROWS ($\uparrow$, $\downarrow$) INDICATE THE PREFERRED DIRECTION FOR EACH METRIC.

Encoder	Cls. (ImageNet-1K)		Count (CLEVR)	Attr. (CLEVR)		Depth (NYU-D v2)	Seg. (Cityscapes)
	Top-1 ($\uparrow\%$)	F1 ($\uparrow\%$)	MAE ($\downarrow$)	Pres. F1 ($\uparrow\%$)	Cnt. MAE ($\downarrow$)	RMSE ($\downarrow$)	mIoU ($\uparrow\%$)
DINOv2	77.28	77.05	7.07	62.50	3.05	0.942	35.3
MAE	47.23	46.71	7.35	62.47	3.12	3.488	22.8
Data2Vec	31.55	30.21	11.01	53.12	3.27	6.518	15.2
Ours (JEPA-based)	73.07	70.95	0.26	98.76	0.27	0.628	43.7

pathway. This baseline isolates the benefit of routing and expert selection.

Pilotless: Disables LoRA specialization and uses the shared base demapper only. This baseline isolates the value of expert adaptation.

Oracle (CSI-known): A label-informed upper bound that evaluates all experts and, for each sample, selects the expert that maximizes the probability of the ground-truth class. This isolates routing error and provides an optimistic upper bound on expert selection.

PRISM-Rx (Ours): Full system with [discrete channel tokenizer \(DCT\)](#), learned routing, LoRA experts, Duo-Track fusion, and early-exit heads.

PRISM-Rx w/o Early-Exit (Ours_no_ee): Ablation that disables early exiting to measure its impact on accuracy and latency.

3) *Training Configuration:* **JEPA Encoder:** We employ a pretrained ViT-H/14 JEPA model from HuggingFace (jmtzt/ijepa_vith14_1k) with 632M parameters. The encoder remains frozen across all experiments. Images are normalized with mean [0.5, 0.5, 0.5] and standard deviation [0.5, 0.5, 0.5] as expected by the pretrained model.

Task Heads (Head Training Phase): Each task head is trained separately with the frozen JEPA encoder. [The classification, depth, counting, attribute, and segmentation heads are each trained for 10 epochs on ImageNet, NYU-D, CLEVR, CLEVR, and Cityscapes, respectively, all using AdamW with a learning rate of \$2e-4\$ and cosine decay to \$2e-5\$. This head training phase establishes baseline task performance on clean features prior to channel-aware training.](#)

PRISM-Rx Training: Stage S0 (tokenizer) trains for 5 epochs in a self-supervised manner using masked-subcarrier prediction and contrastive predictive coding (CPC) on received signals under the channel mixture. For S0, received signals are generated using the mapper initialized with random weights; the tokenizer learns to extract channel characteristics directly from these noisy observations without requiring a pre-trained mapper. Stages S1/S2 freeze the tokenizer and train the mapper, router, LoRA experts, fusion module, and task heads for 20 epochs in Stage S1 followed by 10 epochs in Stage S2 under a CVaR-DRO objective over multiple channel realizations per batch (e.g., $\alpha=0.9$, $K=3$, $\lambda_{\text{tail}}=0.2$), using sharpness-aware minimization (SAM) during the final S2 phase. We use a batch size of 32 per GPU. The SNR is uniformly sampled from [5, 20] dB during training. The

number of experts $N_e = 5$, LoRA rank $r = 8$, codebook size $K = 256$, and I/Q window size $W = 128$. The mapper output dimension is $M = 512$, providing 2.5:1 compression from the JEPA pooled features ($D = 1280$). We set $N_e = 5$ to balance specialization vs. routing/parameter overhead.

Receiver Commissioning on Unseen Channels: For few-shot adaptation experiments, we freeze the tokenizer, base receiver, and existing experts, and adapt only the LoRA parameters of one newly added expert together with the router head. We use AdamW with a learning rate of at most 10^{-3} , batch size 16–32 depending on the shot budget, a cosine-decay schedule with 10% linear warmup, weight decay 5×10^{-4} , gradient clipping at 1.0, and dropout 0.1 on the router MLP. Router regularization matches PRISM-Rx pretraining through the same z-loss weight and load-balancing term.

Channel Mixture: Training samples channel types according to the following weights: AWGN 25%, Rayleigh fading 20%, Rician fading 15%, multipath 25%, and hardware impairments 15%. [These channel realizations are generated by the channel models used in our training pipeline. Multipath channels use 3-7 taps randomly. Rician K-factor is sampled from \[0.5, 5.0\]. Hardware impairments include carrier frequency offset \(CFO\) up to 10 ppm, phase noise standard deviation 0.02 rad, timing offset up to 0.2 samples, IQ imbalance up to 1 dB, and Rapp power-amplifier nonlinearity with \$p = 3\$. Impulsive noise is modeled as sparse high-amplitude outliers superimposed on AWGN, with impulse probability 0.01 and amplitude scaling factor 5.0, representing transient interference common in wireless systems. Impulsive noise is evaluated as an out-of-mixture stress test and is not included in the training mixture.](#)

B. Results and Analysis

1) *Encoder Comparison Across Tasks:* Before evaluating the full communication system, we first compare different pretrained visual encoders to validate our choice of JEPA. We extract features from JEPA-ViT-H/14, DINOv2-Large [13], Masked Autoencoder (MAE)-Large [14], and Data2Vec-Base [15] on clean images, [because these encoders represent widely used contrastive, masked-prediction, and self-distillation pretraining objectives](#), then train lightweight task heads in the head training phase. Table I presents [clean-feature results obtained by attaching lightweight task heads to frozen encoder features rather than end-to-end transmission results across five primary task families.](#)

TABLE II

PRISM-Rx PERFORMANCE ACROSS STANDARD CHANNEL TYPES AND MIXED CONDITIONS. OUR MOE SYSTEM MATCHES OR EXCEEDS PRACTICAL CSI-BLIND BASELINES ACROSS SCENARIOS WHILE MAINTAINING CSI-BLIND OPERATION. METRICS: TOP-1 ACCURACY (Acc1), 10TH PERCENTILE ACCURACY (P10), 5TH PERCENTILE (P05), OUTAGE RATE AT 50% THRESHOLD, AND INFERENCE LATENCY. LATENCY IS MEASURED AS WALL-CLOCK FORWARD-PASS TIME FOR THE RECEIVER STACK UNDER A FIXED EVALUATION SETUP; EARLY-EXIT IS USED AS CONFIDENCE-BASED OUTPUT SELECTION IN OUR CURRENT IMPLEMENTATION. *Mixed (uniform)* SAMPLES ALL STANDARD CHANNEL TYPES UNIFORMLY WITHIN EACH EVALUATION BATCH.

Scenario	Method	Acc1 (%)	P10 (%)	P05 (%)	Outage@0.5	Latency (ms)
AWGN	Mono	68.20	59.38	56.25	0.0160	6.21
	Pilotless	68.70	59.38	56.25	0.0140	6.08
	Oracle	73.10	62.50	59.38	0.0060	30.50
	Ours (no EE)	70.03	59.38	56.25	0.0072	6.19
	Ours	70.20	59.38	56.25	0.0070	6.33
Fading/Multipath	Mono	66.00	56.25	53.13	0.0210	6.22
	Pilotless	66.30	56.25	53.13	0.0190	6.07
	Oracle	70.00	59.38	56.25	0.0080	30.54
	Ours (no EE)	68.23	56.25	53.13	0.0119	6.25
	Ours	69.10	56.25	53.13	0.0100	6.34
Impulsive noise	Mono	65.40	59.38	53.13	0.0220	6.23
	Pilotless	65.70	59.38	53.13	0.0200	6.10
	Oracle	69.60	59.38	56.25	0.0090	30.53
	Ours (no EE)	68.65	59.38	53.13	0.0109	6.17
	Ours	68.70	59.38	53.13	0.0110	6.35
Hardware impairments	Mono	64.90	56.25	53.13	0.0240	6.25
	Pilotless	65.10	56.25	53.13	0.0220	6.12
	Oracle	70.20	62.50	59.38	0.0070	30.47
	Ours (no EE)	69.59	59.38	56.25	0.0072	6.20
	Ours	69.30	59.38	56.25	0.0090	6.36
Mixed (uniform)	Mono	65.64	53.13	50.00	0.0381	6.26
	Pilotless	65.68	53.13	50.00	0.0386	6.08
	Oracle	67.47	56.25	53.13	0.0258	30.70
	Ours (no EE)	65.93	53.13	50.00	0.0374	6.17
	Ours	65.94	53.13	50.00	0.0374	6.27

TABLE III

LATENT FIDELITY UNDER CHANNEL. COSINE SIMILARITY AND LINEAR CKA BETWEEN THE CLEAN POOLED LATENT $\bar{\mathbf{z}}$ AND RECOVERED LATENT $\hat{\mathbf{z}}$ FOR PRISM-Rx, AGGREGATED OVER CHANNEL REALIZATIONS (HIGHER IS BETTER).

Scenario	Cosine Sim. ($\uparrow$)	CKA ($\uparrow$)
AWGN	0.996	0.995
Fading/Multipath	0.992	0.990
Impulsive noise	0.989	0.987
Hardware impairments	0.985	0.982
Mixed conditions	0.990	0.988

JEPA-ViT-H/14 demonstrates strong performance across diverse task families. For classification, DINOv2-Large achieves the highest top-1 accuracy at 77.28% vs JEPA's 73.07%, indicating that contrastive learning excels at discriminative image-level tasks. However, JEPA dramatically outperforms on structured prediction tasks. For object counting, JEPA achieves 0.26 MAE vs 7.07 for DINOv2-Large, a 27 $\times$ improvement, demonstrating superior spatial reasoning. On attribute recognition from CLEVR, JEPA achieves 98.76% presence F1 vs 62.50% for DINOv2-Large (+36.26%), and 0.27 count MAE vs 3.05 (11.3 $\times$ better), showing exceptional fine-grained understanding. On depth estimation (NYU-D v2), JEPA achieves 0.628 RMSE vs 0.942 for DINOv2-Large (30% improvement). Finally, for semantic segmentation on Cityscapes, JEPA achieves 43.7% mIoU vs 35.3% for DINOv2-Large (+8.4% relative gain).

These results reveal JEPA's key strength: masked prediction learns spatially-grounded representations that excel at counting and dense prediction tasks, which are particularly needed for

semantic communication. While DINOv2's contrastive objective produces better discriminative features for classification, JEPA's predictive objective captures richer spatial structure and compositional reasoning. For multi-task semantic transmission, JEPA provides superior overall utility by enabling diverse downstream applications from counting to segmentation and attribute recognition. The 98.76% attribute presence accuracy particularly validates JEPA's fine-grained semantic understanding, crucial for wireless applications requiring detailed scene interpretation.

2) *PRISM-Rx Performance on Standard Channels*: Since SemCast transmits a single task-agnostic representation and downstream tasks operate on the recovered representation, we use ImageNet classification as a representative semantic probe for receiver robustness under channel impairments. We evaluate PRISM-Rx across five channel scenarios representing representative wireless environments: AWGN, fading/multipath, impulsive noise, hardware impairments, and mixed conditions. Each scenario tests 1000 ImageNet-128 samples with SNR uniformly sampled from [5, 20] dB; inputs are then upsampled to 224 $\times$ 224 before the frozen ViT-H/14 encoder to produce $L=196$ tokens. Table II presents classification performance with detailed robustness metrics, where P10 and P05 denote the 10th and 5th percentiles of batch top-1 accuracy across stochastic channel realizations, and outage@0.5 denotes the fraction of realizations whose batch top-1 accuracy falls below 50%.

P10/P05 and outage are computed over a collection of per-batch accuracies from multiple stochastic channel realizations during evaluation. Specifically, for each mini-batch we evaluate independent channel realizations and record the resulting

TABLE IV
COMPARISON OF THE TWO PRISM ADAPTATION MODES ON UNSEEN CHANNELS, INCLUDING THE ZERO-SHOT BASELINE. ACCURACIES ARE TOP-1 ON IMAGENET-1K (PERCENT).

Channel	Shots	Zero-shot	Mode 1: Router	Mode 2: Expert
urban_micro	10	48.7	55.6	54.0
	50	48.7	60.9	62.4
	100	48.7	63.9	65.4
	500	48.7	68.5	70.0
rural	10	47.2	54.8	53.3
	50	47.2	60.1	61.6
	100	47.2	63.2	65.4
	500	47.2	67.0	68.5
high_speed	10	41.9	51.7	50.2
	50	41.9	55.6	57.1
	100	41.9	58.6	60.1
	500	41.9	63.2	64.7

batch top-1 accuracy; P10/P05 are the 10th/5th percentiles of these batch accuracies, and outage@0.5 is the fraction of realizations with batch accuracy below 50%. With batch size 32, batch accuracies are quantized in steps of 1/32 (3.125%), explaining the discrete values in Table II. Across scenarios, PRISM-Rx consistently improves over Mono and Pilotless while remaining close to the Oracle upper bound, with the clearest gains under fading/multipath and hardware impairments where expert specialization is most useful.

a) *Latent fidelity under channel*: To directly quantify the fidelity of latent transmission itself, we also report representation-level alignment between the clean pooled latent $\bar{\mathbf{z}}$ and the recovered receiver latent $\hat{\mathbf{z}}$ under the channel. We compute cosine similarity and linear centered kernel alignment (CKA) between $\hat{\mathbf{z}}$ and $\bar{\mathbf{z}}$ on the same samples and channel realizations. Concretely, for each mini-batch we form $\bar{\mathbf{z}}$ from the frozen JEPA encoder, transmit it through the mapper and channel, recover $\hat{\mathbf{z}}$ via PRISM-Rx, and then compute similarity metrics over the batch. The results in Table III provide direct evidence that PRISM-Rx recovers a unified latent that remains aligned with the transmitted representation under channel impairments.

The consistently high cosine similarity and CKA values show that the recovered latent remains tightly aligned with the transmitted pooled representation across all tested channel families, which is the representation-level basis for the downstream robustness reported above.

3) *Multi-Task Robustness Under Channel Impairments*: We evaluate PRISM-Rx on the full suite of tasks to demonstrate multi-task robustness: object counting and attribute recognition on CLEVR, depth estimation on NYU-D, and semantic segmentation on Cityscapes. For each task, we measure performance under the same five channel scenarios (AWGN, fading/multipath, impulsive noise, hardware impairments, and mixed conditions) with SNR sampled uniformly from [5, 20] dB. The transmitter and PRISM-Rx receiver remain identical across all tasks; only the lightweight task-specific head changes at the receiver. This confirms the core SemCast capability: one transmitted representation serving multiple semantic tasks without reconfiguring the communication pipeline.

a) *Object Counting on CLEVR*: Table V reports mean absolute error (MAE) for counting objects in CLEVR scenes. Lower MAE indicates better performance. The Mono baseline uses a single expert, Pilotless disables LoRA adaptation, and

TABLE V
CLEVR OBJECT COUNTING PERFORMANCE (MAE, LOWER IS BETTER) UNDER DIVERSE CHANNEL CONDITIONS. PRISM-Rx MAINTAINS LOW COUNTING ERROR ACROSS ALL SCENARIOS, WITH PARTICULAR ADVANTAGES UNDER FADING/MULTIPATH AND HARDWARE IMPAIRMENTS WHERE EXPERT SPECIALIZATION COMPENSATES FOR CHANNEL DISTORTION.

Scenario	Mono	Pilotless	Oracle	Ours
AWGN	0.42	0.38	0.31	0.35
Fading/Multipath	0.58	0.52	0.42	0.46
Impulsive noise	0.51	0.47	0.38	0.42
Hardware impairments	0.63	0.57	0.45	0.49
Mixed (uniform)	0.55	0.50	0.43	0.46

TABLE VI
CLEVR ATTRIBUTE RECOGNITION PERFORMANCE (PRESENCE F1, %, HIGHER IS BETTER) UNDER DIVERSE CHANNEL CONDITIONS. THE TASK REQUIRES FINE-GRAINED DISCRIMINATION OF COLOR, SHAPE, AND MATERIAL PROPERTIES, MAKING IT SENSITIVE TO LATENT QUALITY DEGRADATION.

Scenario	Mono	Pilotless	Oracle	Ours
AWGN	94.2	95.1	97.5	96.3
Fading/Multipath	89.7	91.4	95.2	93.8
Impulsive noise	91.5	92.8	96.1	94.7
Hardware impairments	87.3	89.1	94.6	92.4
Mixed (uniform)	90.1	91.6	95.8	93.5

Oracle provides a CSI-aware upper bound.

The counting results confirm that spatial reasoning survives channel impairments when the shared latent is preserved. PRISM-Rx achieves MAE below 0.5 even under challenging hardware impairment and fading conditions, outperforming both the Mono and Pilotless baselines. The gap between PRISM-Rx and the Pilotless baseline (shared demapper without LoRA experts) is particularly pronounced under fading/multipath (+0.06 MAE for Pilotless), confirming that expert adaptation preserves fine-grained spatial information. The Oracle bound remains within 0.1 MAE of PRISM-Rx across all scenarios, showing that the learned router selects appropriate experts without requiring explicit CSI.

b) *Attribute Recognition on CLEVR*: Table VI reports attribute presence F1 score (higher is better) for recognizing fine-grained object properties. This confirms that detailed semantic features survive transmission.

Summary of multi-task robustness. Across counting, attributes, depth, and segmentation, PRISM-Rx consistently outperforms both the Mono single-expert baseline and the Pilotless non-adaptive baseline. The advantages are largest for spatially demanding tasks (depth, segmentation) under severe impairments (fading, hardware), where the flexibility to select different experts for different channel conditions provides measurable gains. The Oracle upper bound remains within reach across all tasks and scenarios, indicating that the learned router approximates optimal expert selection without requiring CSI. These results validate that transmitting one shared JEPA latent, rather than task-specific representations, enables robust multi-task semantic communication.

4) *Few-Shot Adaptation on Unseen Channels*: We next evaluate PRISM-Rx's commissioning-time adaptation capability with unseen channels following Section VI. The base receiver is trained on a mixture of five synthetic channel families (AWGN, Rayleigh, Rician, multipath, and hardware impairments). At test time, we consider three unseen channel scenarios that are not part of this mixture: *urban_micro*, a small-cell scenario; *rural*, a larger-delay-spread rural scenario; and *high_speed*, a high-mobility scenario. We study receiver specialization using a small labeled calibration set of size $k \in \{10, 50, 100, 500\}$ (i.e., *k-shot adaptation*). All experiments use ImageNet-1K classification on 128×128 crops. The transmitter remains frozen; only the receiver is commissioned.

a) *Commissioning setup reference*: The optimization settings for receiver commissioning on unseen channels are summarized above in the Training Configuration subsection; here we focus on the resulting adaptation behavior under each unseen channel scenario.

Across all scenarios, commissioning improves over the zero-shot baseline in Table IV. A 50-shot calibration set already yields several percentage points of Top-1 improvement over zero-shot, and performance continues to improve as more labeled examples become available.

b) *Comparison of adaptation modes*: Table IV compares two inference policies after applying the same expert-expansion + router fine-tuning adaptation procedure, together with the zero-shot baseline. Mode 1 (PRISM-Adapt-Router) uses the learned router's standard top-1 selection at inference, reflecting deployment behavior. Mode 2 (PRISM-Adapt-Expert) reports a forced-expert evaluation that always routes samples to the newly added expert, isolating expert quality from routing errors.

TABLE VII

UNKNOWN-CHANNEL ADAPTATION ON UNSEEN CHANNEL SCENARIOS (TOP-1 ACCURACY ON IMAGE-1K, %). WE COMPARE PRISM ADAPTATION MODES (PRISM-ADAPT-ROUTER, PRISM-ADAPT-EXPERT) AGAINST DIFFUSION-BASED JSCC BASELINES (CDDM, CDM-JSCC, DIFFJSCC) UNDER THE SAME CHANNEL SCENARIOS AND THE SAME ADAPTATION SHOT COUNTS. ALL ENTRIES REPORT DOWNSTREAM TOP-1 CLASSIFICATION ACCURACY (HIGHER IS BETTER).

Channel	Shots	Adapt-Router Acc1 (%)	Adapt-Expert Acc1 (%)	CDDM Acc1 (%)	CDM-JSCC Acc1 (%)	DiffJSCC Acc1 (%)
Urban micro	10	55.6	54.0	29.0	24.1	29.1
	50	60.9	62.4	30.0	25.2	34.7
	100	63.9	65.4	30.5	27.3	36.2
	500	68.5	70.0	31.5	30.1	37.1
Rural	10	54.8	53.3	30.0	27.9	28.7
	50	60.1	61.6	31.0	28.2	30.7
	100	63.2	65.4	31.5	30.1	31.6
	500	67.0	68.5	32.5	31.7	33.4
High speed	10	51.7	50.2	27.0	26.3	30.3
	50	55.6	57.1	28.0	26.5	32.3
	100	58.6	60.1	28.5	26.9	33.5
	500	63.2	64.7	29.5	28.8	33.9

Mode 1 can outperform forced routing in the extreme low-shot regime because the router can still leverage the most compatible pre-existing expert when the newly added expert is undertrained. As more labeled examples become available, the adapted expert improves and the performance of Mode 2 increases accordingly. Mode 1 remains the deployment path because it preserves the standard router-mediated inference

TABLE VIII

ENCODER MODEL SIZES AND LATENT REPRESENTATION OVERHEAD. JEPA-ViT-H/14 ACHIEVES SUPERIOR MULTI-TASK PERFORMANCE ON STRUCTURED PREDICTION TASKS (COUNTING, SEGMENTATION, ATTRIBUTES) DESPITE LARGER PARAMETER COUNT; "ACCURACY-TO-SIZE" REFERS TO TASK PERFORMANCE PER TRANSMITTED DIMENSION AFTER POOLING AND DIMENSIONALITY REDUCTION TO $M = 512$ CHANNEL USES.

Encoder	Parameters (M)	Tokens L	Dim D	Latent Size (KB, float32)
DINOv2-Large	307	196	1024	784
MAE-Large	307	196	1024	784
Data2Vec-Base	86	196	768	588
JEPA-ViT-H/14	632	196	1280	980

behavior over all experts, whereas Mode 2 forces all samples through the specialized expert and is therefore used only as a diagnostic view of expert quality after commissioning.

c) *Comparison with diffusion-based JSCC baselines*:

Finally, we compare PRISM's two unknown-channel adaptation modes against three diffusion-based JSCC baselines, namely CDDM [4], CDM-JSCC [29], and DiffJSCC [30], which are included because they represent diffusion-based semantic transmission pipelines, under the same unseen channel scenarios and the same adaptation shot counts, and present the results in Table VII. We evaluate all methods using the same downstream semantic metric: ImageNet-1K Top-1 classification accuracy (Acc1, %). This aligns the comparison with our goal of robust semantic recovery under deployment shift, and ensures that each row of Table VII corresponds to a matched adaptation regime (same channel scenario, same number of labeled adaptation examples).

Table VII shows that PRISM adaptation substantially outperforms the diffusion-based JSCC baselines across all three unseen channels and all shot budgets. PRISM-Adapt-Expert provides the strongest overall gains, while PRISM-Adapt-Router remains competitive with slightly lower accuracy, reflecting the trade-off between adapting only routing decisions versus adding a new expert for channel-specific specialization. Among the diffusion baselines, DiffJSCC is consistently the strongest, and improves with additional shots, but remains far below PRISM's task accuracy even at 500-shot adaptation. Overall, these results indicate that conditional computation over a CSI-free tokenization pipeline is markedly more effective for preserving discriminative semantics under unknown channels than diffusion-JSCC baselines under the same adaptation budgets.

5) *Complexity and Compression Analysis*:

a) *Encoder size and latent overhead*: Table VIII compares the parameter counts and latent representation sizes for each encoder. The latent size determines transmission overhead before compression.

JEPA-ViT-H/14 has 632M parameters with 980 KB latent size per image ($196 \text{ tokens} \times 1280 \text{ dim} \times 4 \text{ bytes}$). Here $L=196$ corresponds to the 14×14 token grid produced by ViT-H/14 at 224×224 input resolution (our code upsamples ImageNet-128 inputs to 224×224 before the frozen encoder). Although JEPA is larger than DINOv2-Large and MAE-Large,

TABLE IX

COMPUTATIONAL FOOTPRINT COMPARISON FOR IMAGENET-128 CONFIGURATIONS. WE REPORT TRAINABLE AND TOTAL PARAMETERS, TRAINABLE FP32 SIZE, TRAINING SCHEDULE, AND THE MAIN INFERENCE-COMPUTE DRIVER. DIFFUSION-BASED RECEIVERS ADDITIONALLY SCALE WITH THE NUMBER OF DENOISING STEPS AT INFERENCE.

Method	Params (trainable)	Params (total)	FP32 size (trainable)	Training schedule	Inference compute
Conditional Denoising Diffusion Model (CDDM) baseline	542,424,272	542,424,272	2069.18 MiB	Diffusion-model training.	JSCC forward + diffusion sampling.
Conditional Diffusion Model Joint Source Channel Coding (CDM-JSCC) baseline	62,245,059	62,245,059	237.45 MiB	Conditional diffusion training.	Diffusion denoiser (steps-dependent).
Diffusion-based Joint Source Channel Coding (DiffJSCC) baseline	31,189,895	31,189,895	118.98 MiB	Single-stage encoder/decoder training.	Single-pass encoder/decoder.
SemCast	17,562,135	648,326,935	66.99 MiB	Head training, tokenizer Stage S0, and receiver Stages S1/S2.	JEPA forward + single PRISM-Rx forward.

TABLE X

IMPACT OF QUANTIZATION ON JEPA REPRESENTATION QUALITY (1000 IMAGENET VALIDATION SAMPLES).

Variant	Cosine Sim. (↑)	MSE (↓)	CKA (↑)
Int8 weights	0.99658	0.001891	0.99647
Token quantized (int8)	0.99997	0.000014	0.99999

its model size remains on the same order of magnitude as other encoders while delivering substantially stronger multi-task performance. To reduce transmission overhead, the mapper compresses the mean-pooled features (1280-dim) to $M = 512$ channel uses, yielding approximately 2 KB after compression. This represents a 2.5:1 reduction in the transmitted payload (from 1280 floats ≈ 5 KB to 512 floats ≈ 2 KB). With int8 quantization, the transmitted payload further reduces to approximately 0.5 KB.

b) System footprint and compute drivers.: We compare computational footprint for ImageNet-128 configurations. By computational footprint we mean the trainable and total parameter counts, the trainable FP32 size, the training schedule, and the main inference-compute driver. Parameter counts are for model weights only (excluding optimizer states); 32-bit floating point (FP32) size is parameters $\times$ 4 bytes. For PRISM-Rx, “total” includes the frozen JEPA backbone.

Table IX shows that SemCast has the smallest trainable footprint among the compared systems because the large JEPA backbone remains frozen, while inference is driven by one JEPA forward pass and one PRISM-Rx forward pass rather than iterative diffusion sampling.

c) Quantization and compression impact.: We analyze how quantization changes the JEPA representation in a frozen setting. Table X reports representation similarity across 1000 ImageNet validation samples using cosine similarity, mean squared error (MSE), and linear centered kernel alignment (CKA). The near-identity similarity values show that low-bit quantization preserves the transmitted representation with very small distortion in the frozen setting.

Model weight quantization to int8 yields cosine similarity 0.99658 and CKA 0.99647, indicating modest distortion in

the pooled representation. Token-level int8 quantization and dequantization yield near-identity similarity (cosine 0.99997 and CKA 0.99999), suggesting that per-sample dynamic-range token quantization largely preserves the pooled representation.

VIII. CONCLUSION

We presented SemCast, a task-oriented semantic communication system that sends one shared JEPA representation for multiple vision tasks and uses PRISM-Rx to perform pilotless, CSI-free reception through tokenization, mixture-of-experts routing, and LoRA specialization. By decoupling representation learning from channel transmission and enabling parameter-efficient commissioning on new channel families, SemCast avoids task-specific encoders and expensive retraining while remaining robust under diverse wireless and hardware impairments. Together with learned UEP and lightweight quantization, the system is practical for edge deployment, delivering reliable downstream predictions and strong latent recovery from a single transmission across heterogeneous channels.

REFERENCES

- [1] J. G. Proakis and M. Salehi, *Digital communications*. McGraw-hill New York, 2001, vol. 4.
- [2] B. Güler, A. Yener, and A. Swami, “The semantic communication game,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 4, no. 4, pp. 787–802, 2018.
- [3] E. Bourtsoulatz, D. B. Kurka, and D. Gündüz, “Deep joint source-channel coding for wireless image transmission,” *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1349–1361, 2019.
- [4] T. Wu, Z. Chen, D. He, L. Qian, Y. Xu, M. Tao, and W. Zhang, “Cddm: Channel denoising diffusion models for wireless semantic communications,” *IEEE Transactions on Wireless Communications*, vol. 23, no. 9, pp. 11 168–11 183, 2024.
- [5] D. B. Kurka and D. Gündüz, “Deepjssc-f: Deep joint source-channel coding of images with feedback,” *IEEE Journal on Selected Areas in Information Theory*, vol. 1, no. 1, pp. 178–193, 2020.
- [6] Q. Pan, H. Tong, J. Lv, T. Luo, Z. Zhang, C. Yin, and J. Li, “Image segmentation semantic communication over internet of vehicles,” in *2023 IEEE Wireless Communications and Networking Conference (WCNC)*, 2023, pp. 1–6.
- [7] H. Xie, Z. Qin, G. Y. Li, and B.-H. Juang, “Deep learning enabled semantic communication systems,” *IEEE Transactions on Signal Processing*, vol. 69, p. 2663–2675, 2021.
- [8] M. Jankowski, D. Gündüz, and K. Mikołajczyk, “Wireless image retrieval at the edge,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 89–100, 2020.

- [9] D. B. Kurka and D. Gündüz, "Successive refinement of images with deep joint source-channel coding," in *2019 IEEE 20th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*. IEEE, 2019, pp. 1–5.
- [10] D. B. Kurka and D. Gündüz, "Bandwidth-agile image transmission with deep joint source-channel coding," *IEEE Transactions on Wireless Communications*, vol. 20, no. 12, pp. 8081–8095, 2021.
- [11] S. Yun, J. Pei, and P. Wang, "Toast: Task-oriented adaptive semantic transmission over dynamic wireless environments," *IEEE Transactions on Network Science and Engineering*, pp. 1–19, 2025.
- [12] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15 619–15 629.
- [13] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.
- [14] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 15 979–15 988.
- [15] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, "Data2vec: A general framework for self-supervised learning in speech, vision and language," in *International Conference on Machine Learning*. PMLR, 2022, pp. 1298–1312.
- [16] R. Singh, "Multiple access techniques for 4g mobile wireless networks," *International Journal of Engineering Research and Development*, vol. 5, no. 11, pp. 86–94, 2013.
- [17] M. Soltani, V. Pourahmadi, A. Mirzaei, and H. Sheikhzadeh, "Deep learning-based channel estimation," *IEEE Communications Letters*, vol. 23, no. 4, pp. 652–655, 2019.
- [18] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 119–131, 2018.
- [19] T. O'shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, 2017.
- [20] S. Park, O. Simeone, and J. Kang, "Meta-learning to communicate: Fast end-to-end training for fading channels," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 5075–5079.
- [21] J. Beck, M. T. Jackson, R. Vuorio, and S. Whiteson, "Hypernetworks in meta-reinforcement learning," in *Conference on Robot Learning*. PMLR, 2023, pp. 1478–1487.
- [22] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, "Scaling vision with sparse mixture of experts," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.
- [23] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6309–6318.
- [24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [26] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, "Indoor segmentation and support inference from rgb-d images," in *European Conference on Computer Vision (ECCV)*, 2012, pp. 746–760.
- [27] J. Johnson, B. Hariharan, L. van der Maaten, J. Hoffmann, L. Fei-Fei, C. Lawrence Zitnick, and R. Girshick, "Clevr: A diagnostic dataset for compositional language and elementary visual reasoning," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2901–2910.
- [28] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3213–3223.
- [29] P. Yang, G. Zhang, and Y. Cai, "Rate-adaptive generative semantic communication using conditional diffusion models," *arXiv preprint arXiv:2409.02597*, 2024.
- [30] M. Yang, B. Liu, B. Wang, and H.-S. Kim, "Diffusion-aided joint source channel coding for high realism wireless image transmission," *arXiv preprint arXiv:2404.17736*, 2024.